Отчет по домашнему заданию №3

Выполнила: Левандо Ксения Андреевна

Этап 1. Поднимаем RustFS (аналогично дз2)

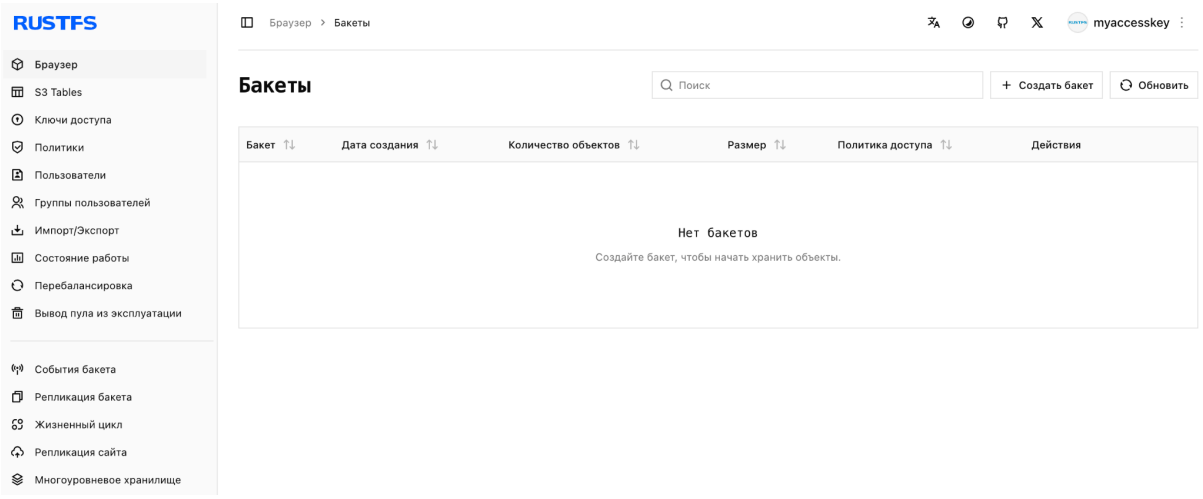
Папки остались с прошлого дз, осталось только поднять через docker run -d

Проверим что поднялось через docker ps --filter name=rustfs

```
CONTAINER ID   IMAGE                     COMMAND                  CREATED          S
TATUS          PORTS
NAMES
d25c6a2c1770   rustfs/rustfs:latest      "/entrypoint.sh rust…"   4 seconds ago    U
p 4 seconds    0.0.0.0:9000-9001->9000-9001/tcp, [::]:9000-9001->9000-9001/tcp
rustfs
```

Поднялось

По http://localhost:9001 вновь видим тот же интерфейс



Этап 2 . Подготовка и загрузка датасета

2.1. Подготовка

Примечание: подготовка датасета проходит в отдельном ноутбуке build_dataset.ipynb

За основу берем датасет it_vacancies_2024.csv на 229388 строк и 23 колонки.

```
df.shape
✓ 0.0s
(229388, 23)
```

В соответствии с моим вариантом, мне предстоит поработать с подвыборкой вакансий специалистов C и C++.

ИИИ-3-ИнжДан-1-3	91193	20	Левандо Ксения Андреевна	6. Программист, разработчик на C и C++

Нужно определить стратегию выделения подвыборки. Ключевыми колонками для нас станут ‘role’ и ‘name’. Посмотрим уникальные значения ролей:

```
df['role'].unique()
✓ 0.0s
<StringArray>
[                    'Программист, разработчик',
                           'Системный аналитик',
                              'Сетевой инженер',
                                    'Аналитик',
                             'Бизнес-аналитик',
    'Специалист по информационной безопасности',
                             'DevOps-инженер',
                      'Системный администратор',
                         'Технический писатель',
                                  'Тестировщик',
                           'Системный инженер',
                             'Дата-сайентист',
                 'BI-аналитик, аналитик данных',
               'Руководитель группы разработки',
                          'Продуктовый аналитик',
 'Директор по информационным технологиям (CIO)',
                 'Руководитель отдела аналитики']
Length: 17, dtype: str
```

Помимо “Программист, разработчик”, прямо указанного в задании, на мой взгляд, стоит добавить к подходящим значениям еще и “Руководитель отдела разработки”.

Отсортировать по соответствию этим 2 возможным значениям, нам недостаточно - эти категории слишком широкие и включают вообще всех разработчиков на всех языках. Напишем скрипт, выделяющий подвыборку.

Первая попытка написания скрипта с поиском только по ‘name’ дала довольно маленькую относительно ориентировочного примера выборку - всего 2125 вакансии. В связи с этим было принято решение расширить условие сбора данных и подключить колонку description.

```
# Итоговая логика:
# (Ищем C++ в name ИЛИ description)
# И (НЕ C# в description)
# И (НЕ другие ЯП в name)
```

Итого у нас получилось 5653 вакансий. Отладочный вывод топ-12 наименований вакансий помогает убедиться, что все в порядке. Также видим, что из роли руководителя удалось выцепить еще 250+ вакансий в выборку, что хорошо для лучшего анализа.

```
Всего вакансий: 229388
Вакансий с нужными ролями: 93407

Найдено вакансий C/C++: 5653

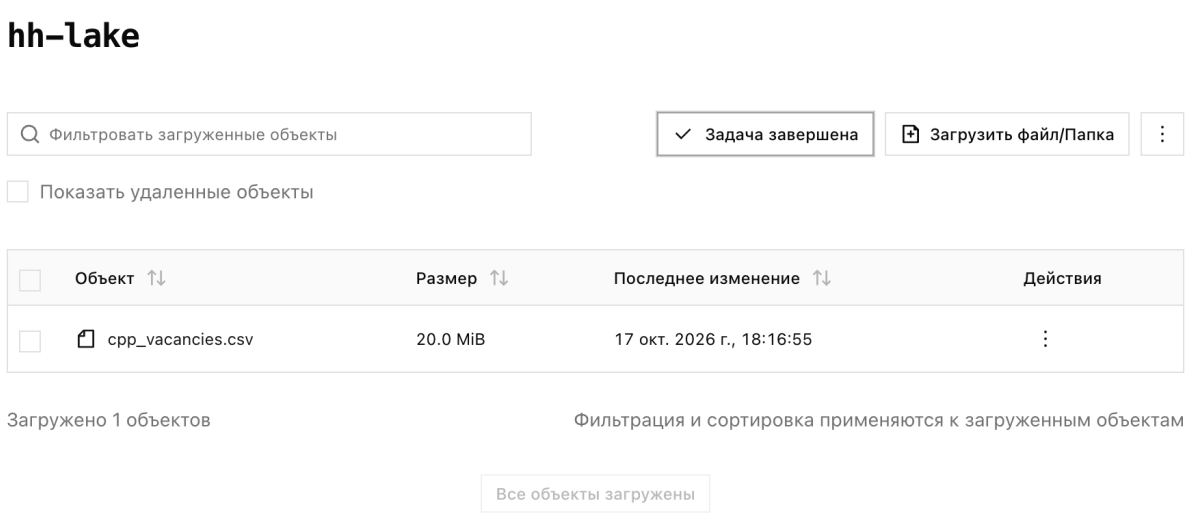
Топ-12 названий найденных вакансий:
name
Инженер-программист                          202
Программист C++                              111
Программист C/C++                             85
Инженер-программист микроконтроллеров         78
Разработчик C++                               74
Программист микроконтроллеров                 66
Разработчик C++                               61
Программист C++                               53
Ведущий инженер-программист                   48
Программист                                   42
Инженер-программист встраиваемых систем       41
C++ Developer                                 38

Распределение по ролям:
role
Программист, разработчик           5377
Руководитель группы разработки      276
```

Сохраняем этот датасет в cpp_vacancies.csv для дальнейшей работы.

2.2. Загрузка

Теперь нам необходимо загрузить этот файл в бакет. Открываем веб-консоль RustFS (http://localhost:9001), заходим в новый бакет (назван hh-lake по условию), и загружаем туда новый файл cpp_vacancies.csv.



Этап 3. Подсчёт навыков

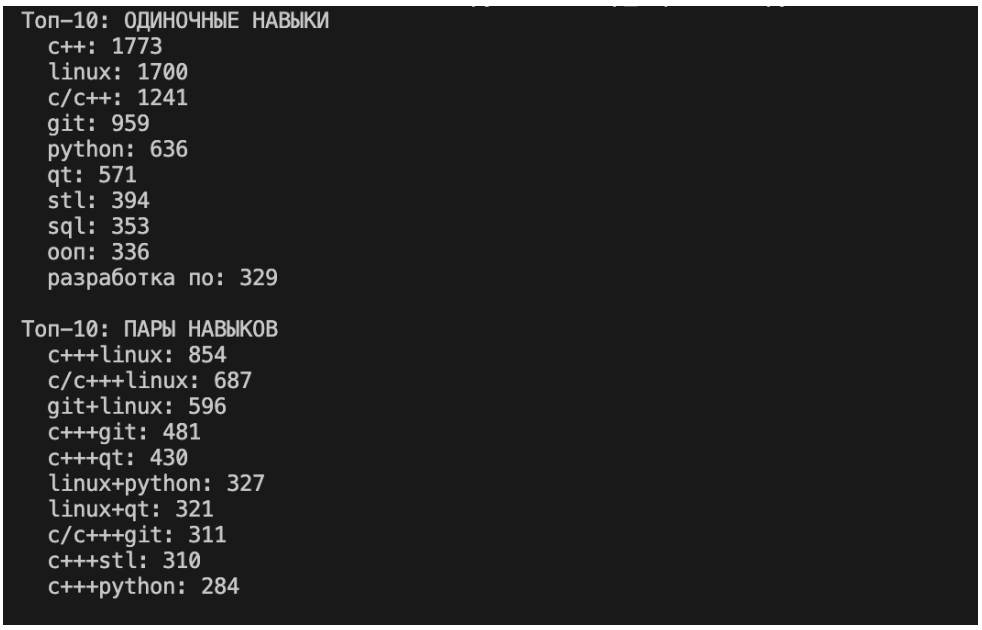
Итак, датасет загружен в объектное хранилище RustFS, его размер составил 20 МБ.

Расчёт частот одиночных навыков, пар и троек был выполнен двумя независимыми способами.

Способ 1. Собственная реализация MapReduce

Для реализации MapReduce был написан скрипт step_mapreduce.py, который читает файл из RustFS частями (чанками). Размер сплита был установлен в 4 МБ, благодаря чему файл размером 20 МБ был разбит на 5 чанков, которые обрабатывались параллельно в 5 воркерах. Внутри каждого воркера mapper.py извлекал навыки, а reducer_stats.py суммировал частоты.

Первая попытка, а именно ее результаты, натолкнули на интересную мысль:



Заметим, что в список навыков попадали сами названия ключевого языка программирования (C, C++, C/C++, CPP). Поскольку анализируются исключительно вакансии C/C++ разработчиков, эти «навыки» являются тавтологией и не несут аналитической ценности.

На основе этого было принято решение исключить названия самого языка из списка навыков на этапе парсинга (как в mapper.py, так и в SQL-запросах DuckDB). Это позволило очистить топы и выявить реальные сопутствующие технологии (Linux, Git, Qt, Boost и др.), а также сохранить полную сходимость результатов между двумя методами.

Результаты с исправленными скриптами уже намного интереснее:

```
Файл загружен: /Users/xtevando/Downloads/hw3/cpp_vacancies.csv (20.0 MB)
Чанков: 5, редьюсеров: 8, воркеров: 5
Время MapReduce: 0.29 сек
Промежуточных записей: 328,081

Топ-10: ОДИНОЧНЫЕ НАВЫКИ
  linux: 1700
  git: 959
  python: 636
  qt: 571
  stl: 394
  sql: 353
  ооп: 336
  разработка по: 329
  postgresql: 303
  английский язык: 245

Топ-10: ПАРЫ НАВЫКОВ
  git&linux: 596
  linux&python: 327
  linux&qt: 321
  git&qt: 225
  linux&stl: 216
  linux&tcp/ip: 187
  git&python: 182
  linux&ооп: 164
  bash&linux: 162
  git&ооп: 162

Топ-10: ТРОЙКИ НАВЫКОВ
  git&linux&qt: 149
  git&linux&python: 124
  git&linux&ооп: 98
  git&linux&postgresql: 95
```

Результаты (Топ-10):

- Одиночные навыки: linux (1700), git (959), python (636), qt (571), stl (394), sql (353), ооп (336), разработка по (329), postgresql (303), английский язык (245).
  (Примечание: сам язык C++ был исключён из списка навыков на этапе парсинга как тавтологичный для данной выборки).
- Пары навыков: git & linux (596), linux & python (327), linux & qt (321), git & qt (225), linux & stl (216).
- Тройки навыков: git & linux & qt (149), git & linux & python (124), git & linux & ооп (98).

Метрики производительности: Время выполнения MapReduce составило 0.29 сек. Было сгенерировано 328 081 промежуточных записей.

Объяснение роста числа комбинаций (закон сочетаний):
Рост количества пар и троек навыков объясняется комбинаторикой. Если в вакансии указано n уникальных скиллов, то количество возможных пар рассчитывается как число сочетаний из n по 2: C^2_n=n(n−1)/2.

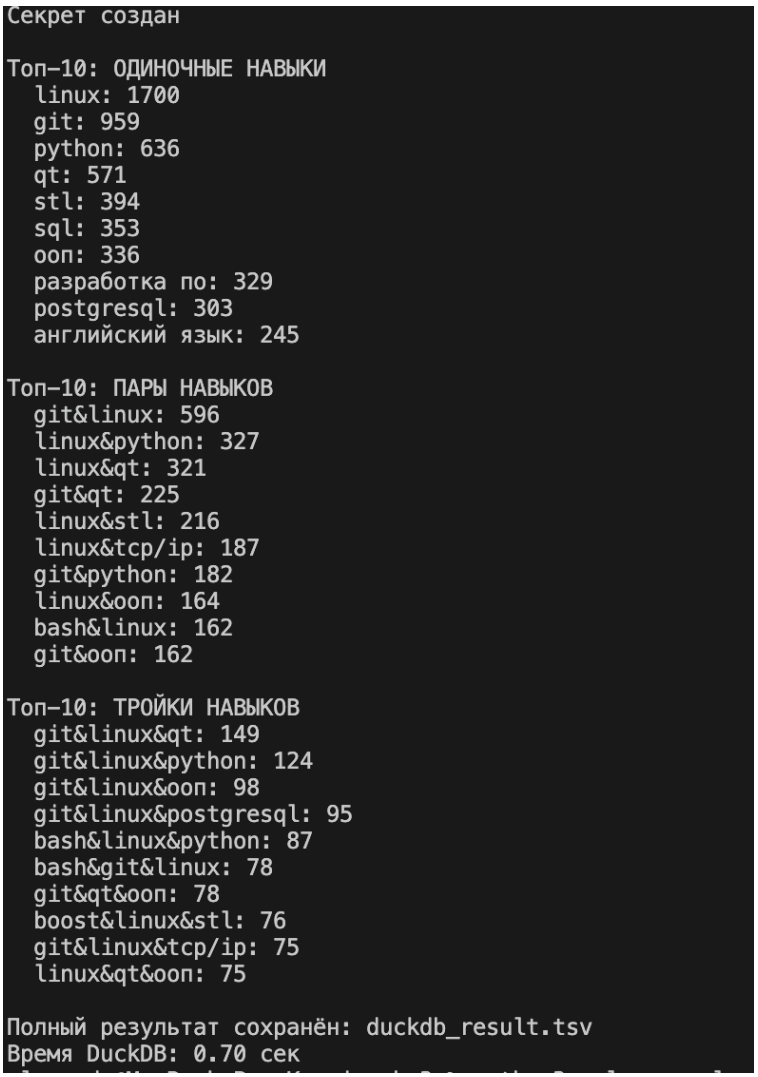
А количество троек - как число сочетаний из n по 3: C^3_n=n(n−1)(n−2)/6

На практике средний стек C++ разработчика содержит от 4 до 7 ключевых навыков (как увидим ниже, см. анализ этапа 5, примерный типичный набор: c++, linux, git, qt, stl). При n=5 мы получаем 10 пар и 10 троек на одну вакансию. При n=7 - уже 21 пару и 35 троек. Именно поэтому при генерации комбинаций количество промежуточных записей в MapReduce возрастает экспоненциально (достигая сотен тысяч), что наглядно демонстрирует вычислительную нагрузку на пайплайн.

Способ 2. SQL-запрос к DuckDB

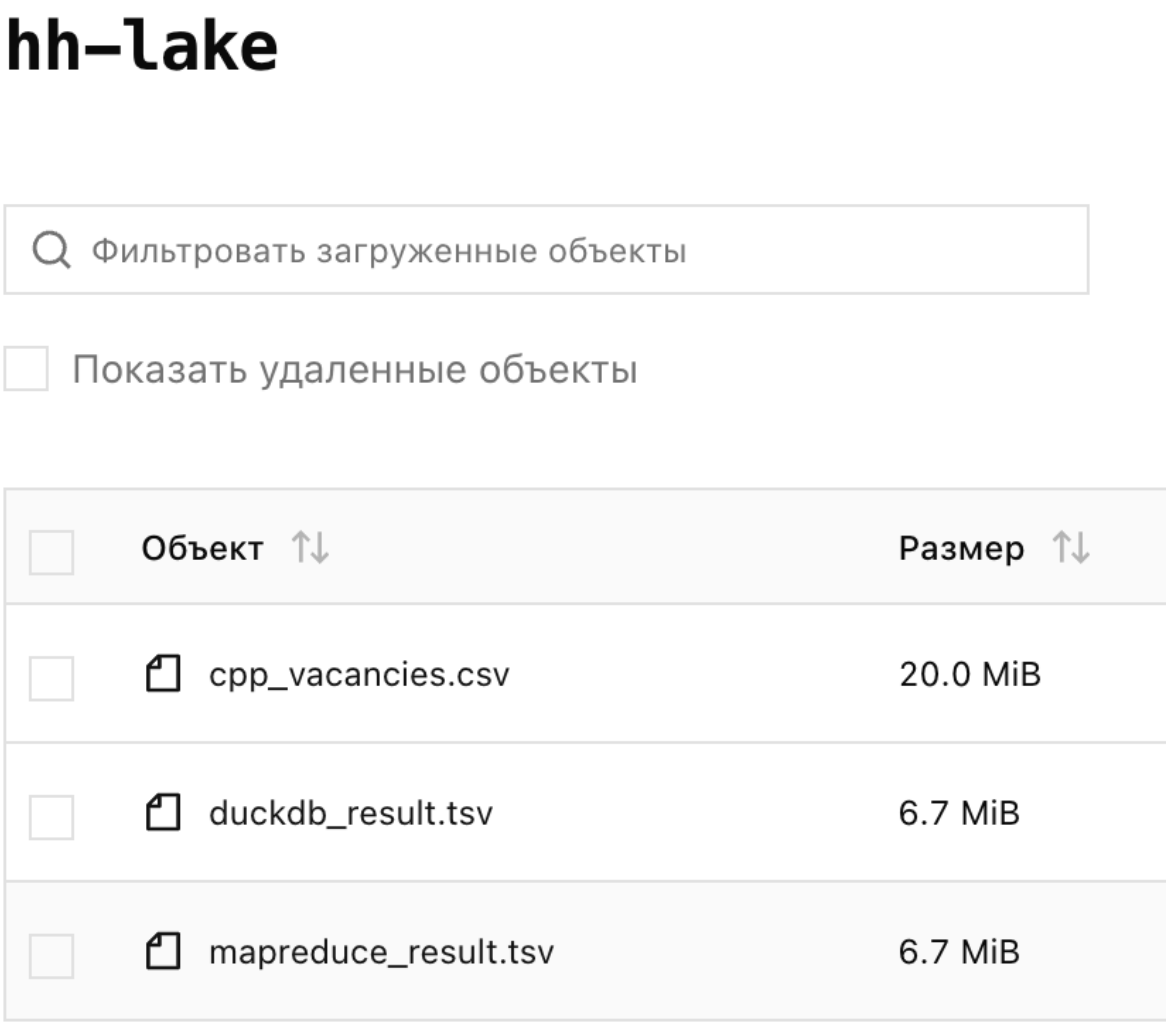
Второй способ реализован в скрипте duckdb_analysis.py. DuckDB подключается к RustFS через S3 API (расширение httpfs) и выполняет SQL-запросы напрямую к файлу, используя UNNEST(string_split(...)) для разбивки навыков и JOIN для поиска пар и троек.

Результаты полностью совпадают с результатами MapReduce (числа идентичны).



Метрики производительности: Время выполнения DuckDB составило 0.70 сек.

Согласно условию, результаты для трёх размеров стека положены в хранилище.



Этап 4. Верификация результатов

Как было показано и отмечено выше, результаты обоих способов полностью совпадают для всех трёх типов расчётов (одиночные навыки, пары, тройки). Это подтверждает, что реализация MapReduce логически эквивалентна SQL-запросам DuckDB, а механизм shuffle и reduce отработал без потерь данных.

Этап 5. Анализ влияния навыков на заработную плату

Для анализа зарплат из датасета были отобраны вакансии с указанной зарплатой в рублях (RUR). Общая выборка составила 1229 вакансий. Общая медианная зарплата для C/C++ разработчиков составила 150 000 руб..

Чтобы найти самые «дорогие» и «дешёвые» комбинации, для каждой пары и тройки навыков была рассчитана медианная зарплата (с порогом вхождения не менее 30 для пар и 20 для троек).

```
C/C++ разработчиков с зарплатой RUR: 1229
 Общая медиана: 150000 руб

 ПАРЫ:
   Найдено: 35
 ТРОЙКИ
   Найдено: 38

  ТОП-10 ПАР: ПОВЫШАЮТ

  python & sql                                n=33    med=   300000 (+150000, +100.0%)
  linux & python                              n=95    med=   200000 (+50000, +33.3%)
  git & postgresql                            n=59    med=   200000 (+50000, +33.3%)
  boost & c++                                 n=38    med=   200000 (+50000, +33.3%)
  boost & linux                               n=31    med=   200000 (+50000, +33.3%)
  boost & git                                 n=38    med=   200000 (+50000, +33.3%)
  git & python                                n=65    med=   200000 (+50000, +33.3%)
  boost & cmake                               n=32    med=   200000 (+50000, +33.3%)
  linux & postgresql                          n=59    med=   200000 (+50000, +33.3%)
  cmake & git                                 n=48    med=   200000 (+50000, +33.3%)

  ТОП-10 ПАР: ПОНИЖАЮТ

  qml & qt                                    n=30    med=    95000 (-55000, -36.7%)
  bash & linux                                n=51    med=    95000 (-55000, -36.7%)
  linux & разработка по                       n=61    med=   100000 (-50000, -33.3%)
  git & stm32                                 n=30    med=   100000 (-50000, -33.3%)
  linux & английский язык                     n=60    med=   100000 (-50000, -33.3%)
  qt & sql                                    n=32    med=   110000 (-40000, -26.7%)
  arm & linux                                 n=45    med=   120000 (-30000, -20.0%)
  arm & c/c++                                 n=48    med=   120000 (-30000, -20.0%)
  git & английский язык                       n=37    med=   120000 (-30000, -20.0%)
  linux & ооп                                 n=79    med=   120000 (-30000, -20.0%)
```

Пары навыков, повышающие зарплату

Пара навыков	**Кол-во**	**Медиана**	**Отклонение от общей**
python & sql	33	300 000 руб.	+100.0%
linux & python	95	200 000 руб.	+33.3%
git & postgresql	59	200 000 руб.	+33.3%

boost & linux	31	200 000 руб.	+33.3%
boost & git	38	200 000 руб.	+33.3%

Вывод: Наибольший прирост к зарплате даёт пересечение C++ с бэкенд-разработкой и базами данных (python & sql, postgresql). Использование библиотеки boost также является маркером высокооплачиваемых вакансий (вероятно, из сфер High-Load и финтеха).

Пары навыков, понижающие зарплату

Пара навыков	Кол-во	Медиана	Отклонение от общей
qml & qt	30	95 000 руб.	-36.7%
bash & linux	51	95 000 руб.	-36.7%
git & stm32	30	100 000 руб.	-33.3%
linux & английский язык	60	100 000 руб.	-33.3%
qt & sql	32	110 000 руб.	-26.7%

Вывод: Более низкие зарплаты характерны для десктопной разработки (qml & qt), встраиваемых систем (микроконтроллеры stm32, arm) и системного администрирования (bash).

Тройки навыков

Самыми высокооплачиваемыми тройками стали комбинации, объединяющие C++ с Python и SQL: linux & python & sql и git & python & sql - их медиана достигает 300 000 руб. (+100%) (git & postgresql & sql, git & postgresql & python , имеющие такую же медиану, несут по сути такой же практический смысл).
Самые низкие зарплаты у троек, связанных с Qt и Embedded: arm & linux & английский язык (95 000 руб., -36.7%).

```
ТОП-10 ТРОЕК: ПОВЫШАЮТ

git & postgresql & sql                    n=32    med=   300000 (+150000, +100.0%)
git & postgresql & python                 n=22    med=   300000 (+150000, +100.0%)
linux & python & sql                      n=25    med=   300000 (+150000, +100.0%)
git & python & sql                        n=24    med=   300000 (+150000, +100.0%)
linux & postgresql & sql                  n=34    med=   300000 (+150000, +100.0%)
linux & postgresql & python               n=21    med=   300000 (+150000, +100.0%)
postgresql & python & sql                 n=23    med=   300000 (+150000, +100.0%)
git & linux & python                      n=53    med=   220000 (+70000, +46.7%)
boost & qt & ооп                          n=20    med=   200000 (+50000, +33.3%)
boost & git & ооп                         n=20    med=   200000 (+50000, +33.3%)

ТОП-10 ТРОЕК: ПОНИЖАЮТ

arm & linux & английский язык             n=22    med=    95000 (-55000, -36.7%)
linux & qml & qt                          n=20    med=    95000 (-55000, -36.7%)
linux & qt & английский язык              n=24    med=    97500 (-52500, -35.0%)
bash & git & linux                        n=22    med=   110000 (-40000, -26.7%)
linux & qt & sql                          n=23    med=   110000 (-40000, -26.7%)
git & linux & ооп                         n=45    med=   120000 (-30000, -20.0%)
linux & qt & разработка по                n=31    med=   120000 (-30000, -20.0%)
git & linux & английский язык             n=27    med=   120000 (-30000, -20.0%)
linux & qt & ооп                          n=59    med=   120000 (-30000, -20.0%)
git & linux & разработка по               n=32    med=   140000 (-10000, -6.7%)
```

Этап 6. Региональный анализ (Этап 6)

Для сравнения были выбраны три топ-региона по кол-ву вакансий: Москва, Санкт-Петербург и Нижний Новгород. Эти города представляют разные уровни масштаба IT-рынка: столица, вторая столица и крупный региональный центр.

Сводная таблица по регионам

Регион	**Всего вакансий**	**С указанной зарплатой**	**Медианная зарплата (RUR)**
Москва	2632	690	180 000 руб.

Санкт-Петербург	957	326	150 000 руб.
Нижний Новгород	237	46	150 000 руб.

Анализ навыков по регионам

Москва

- Топ-5 навыков: linux (793), git (424), python (326), qt (245), sql (184).
- Топ-3 пары: git & linux (260), linux & python (165), linux & qt (136).
- Заметим наличие sql и python в топе. Это указывает на то, что в Москве много вакансий на стыке C++ с бэкенд-разработкой и базами данных. Это коррелирует с более высокой медианной зарплатой (180 000 руб.).

Санкт-Петербург

- Топ-5 навыков: linux (306), git (161), qt (112), python (106), stl (71).
- Топ-3 пары: git & linux (103), linux & qt (62), linux & python (44).
- Любопытно, что медиана зарплаты (150 000 руб.) ниже московской на 16.7%. В топе навыков присутствует stl, что указывает на более классическую C++ разработку (алгоритмы, системное программирование).

Нижний Новгород

- Топ-5 навыков: linux (96), git (50), python (42), qt (23), stl (23).
- Топ-3 пары: git & linux (28), linux & python (26).
- Интересно, что несмотря на меньшее количество вакансий (237), медианная зарплата в Нижнем Новгороде (150 000 руб.) догнала Санкт-Петербург. Стек навыков похож, но с меньшим перевесом в сторону qt. Это говорит о высоком уровне локальных IT-компаний.

Выводы по региональному анализу

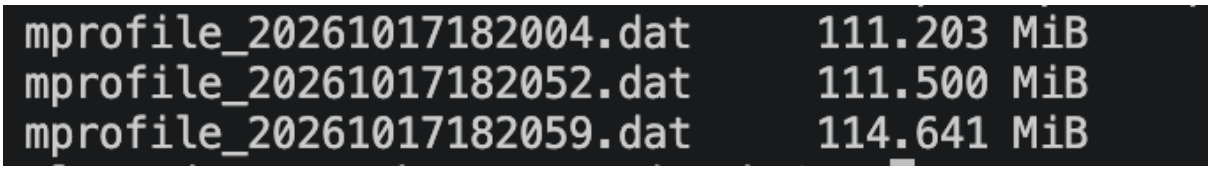
1. Медианная зарплата: Москва (180 000 руб.) > Санкт-Петербург (150 000 руб.) = Нижний Новгород (150 000 руб.).

2. Типичный стек: в Москве и Питере доминируют linux, git, python и qt. В Нижнем Новгороде топ-5 почти идентичен, но с меньшим количеством sql - вероятно, из-за специфики рода деятельности местных компаний (промышленная автоматизация, embedded).
3. Востребованность: связка git & linux является абсолютным лидером во всех трёх регионах, что подтверждает её статус базового требования для C/C++ разработчика.

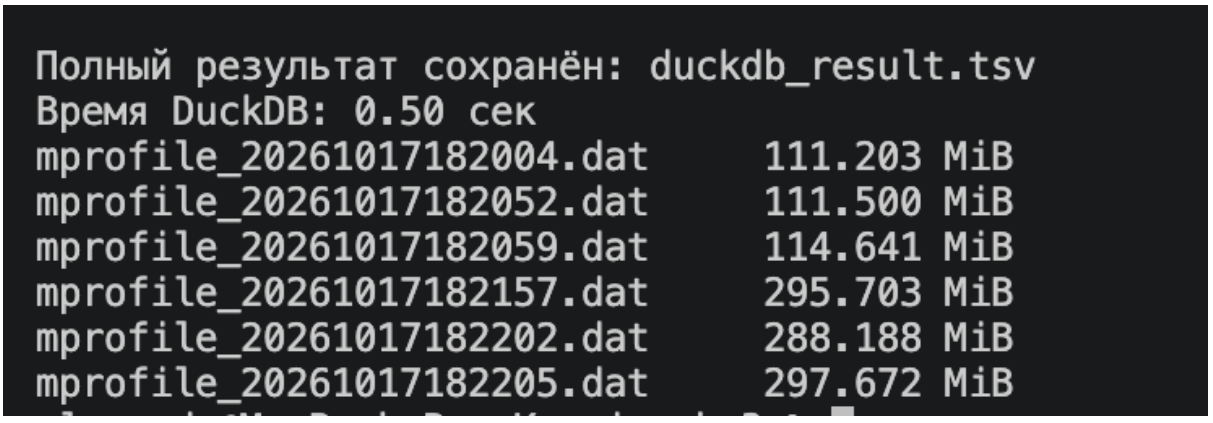
Этап 7*. Профилирование

Для оценки эффективности двух подходов (MapReduce и DuckDB) было проведено профилирование с помощью утилиты memory_profiler. Было выполнено по три независимых запуска для каждого метода, в ходе которых замерялось время выполнения и пиковое потребление оперативной памяти.

Результаты замеров:



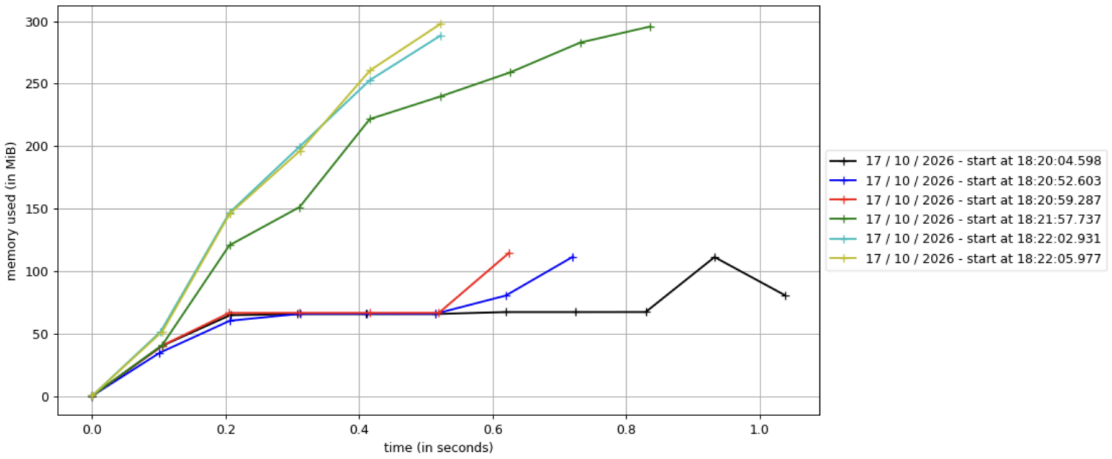
Скриншот после 3 запусков MapReduce



Скриншот после 3+3 запусков обоих способов

Сводная таблица:

Попытка	Время MapReduce	Пик памяти MapReduce	Время DuckDB	Пик памяти DuckDB
1	0.31 сек	111.2 МБ	0.68 сек	295.7 МБ
2	0.3 сек	111.5 МБ	0.52 сек	288.2 МБ
3	0.29 сек	114.6 МБ	0.5 сек	297.7 МБ



Общих график всех 6 запусков

Анализ графиков потребления памяти:

1. Скорость работы: реализация MapReduce стабильно работает где-то чуть менее, а где-то и более чем в 2 раза быстрее. Это логично, так как DuckDB тратит дополнительное время на инициализацию соединения с объектным хранилищем RustFS, загрузку расширения httpfs и выполнение сложных SQL-операций JOIN (особенно для поиска троек навыков).
2. Потребление памяти: MapReduce требует примерно в 2.6 раза меньше памяти. Как видно на графике, потребление памяти в MapReduce (нижние линии) остается стабильным и низким, так как данные обрабатываются чанками (по 4 МБ) и не загружаются в память целиком.
   DuckDB (верхние линии) показывает значительно более высокий пик памяти, что связано с необходимостью держать в оперативной памяти промежуточные таблицы для выполнения тройных JOIN’ов.

3. Стабильность: разброс значений по памяти минимален (в пределах 5-10 МБ от запуска к запуску), что говорит о воспроизводимости результатов и отсутствии утечек памяти.
4. Масштабируемость: реализация MapReduce лучше подходит для обработки больших объемов данных, так как позволяет горизонтально масштабировать вычисления (добавлять новые воркеры), не увеличивая потребление памяти на одной машине.